From Paper to Proof

Session 3: AI tools

Michail Karatarakis

17 November 2026

BGSMath Graduate Course · UPC, Barcelona

The course

	Session	Content
1	Lean and Mathlib	What formalizing means, and does not mean; first proofs; finding things in Mathlib; reading a Lean statement
2	From paper to Lean	Writing definitions and statements; the usual mistakes; drafting statements with AI and checking them
3	AI tools	What AI did this year; an agent proving a statement, live; checking what it hands back
4	Your own theorem	Statements sent in by participants, formalized live; how to do your own at home

Each session: a lecture with live demonstrations in Lean. You redo them, and the exercises, at home.

Last time, and today

> Lean checks the **proof**. You check the **statement**.

Last time: writing

- Definitions and statements, and tests for them.
- A statement you trust, ending in `sorry`.

Today: proving, with AI

- What the tools did this year.
- An agent proving a statement, live.
- Checking what comes back.

What happened this year

Fermat's Last Theorem

- Claude "worked largely autonomously over 11 days": **13 million lines** of Lean, with 29,500 intermediate theorems.
- It followed a simplified version of Wiles's proof, and adapted pieces of Buzzard's FLT project and of `flt-regular`.
- The agents ran on Prove2Me and a multi-agent harness built on Claude Code.
- "Mathematical input from humans was limited to occasional high-level instructions [...]"

How the result was checked

"The finished proof was checked by Lean; it uses just Lean's three standard axioms, and a comparator confirmed that the theorem's statement matches Mathlib's own statement of FLT."

Anthropic, *Formalizing Fermat's Last Theorem*, 4 September 2026.

The unit distance conjecture

20 May	ChatGPT disproves Erdős's unit distance conjecture.
26 May	An autoformalization of the whole paper, *conditional* on two classical theorems not in Mathlib (Golod–Shafarevich, and a theorem of Shafarevich).
26 June	B. Alexeev, with OpenAI's model Sol: a complete formalization, "assuming nothing beyond the axioms of mathematics". 1.2 million lines, in three weeks.

> **The statements, again** In the conditional version, K. Buzzard "found errors in the statements of the external theorems": the theorems that were assumed, not proved.

K. Buzzard, Xena blog, 20 July 2026; Logical Intelligence, *Aleph Prover* blog post.

Gauss: sphere packing and the prime number theorem

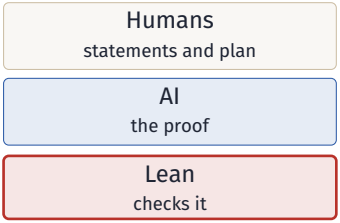
- **Strong prime number theorem.** A challenge set by Tao and Kontorovich in January 2024. Math Inc's agent Gauss completed it "after three weeks of effort": about 25,000 lines.
- **Sphere packing.** A project started by Hariharan and Viazovska in 2024. According to Math Inc, Gauss proved the remaining results for dimension 8 "in just 5 days", then formalized dimension 24 in two weeks, "using only the original paper as input".
- **Access.** Gauss is in early access, for a select group of mathematicians.

Not everyone tells it the same way A guest post on the Xena blog (16 October 2026) gives a critical account of Gauss's part in the sphere packing project. Read both.

Math, Inc., *Introducing Gauss* and *Sphere packing*; Xena blog, 16 October 2026.

What these have in common

- The **statement** was fixed before the proof: Mathlib's FLT, the paper's theorem, a human blueprint.
- The AI wrote the **proof**: millions of lines that nobody reads.
- Lean, and a comparator, did the **check**.
- Where things went wrong, they went wrong in the statements.



> The same division of labour works for one theorem of yours.

The tools

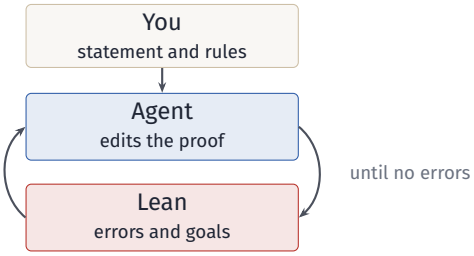
The tools, November 2026

Tool	What it is	Can you use it?
Claude Code	A coding agent: works in your Lean project, runs Lean, reads the errors, edits, repeats.	Yes, with a paid plan.
Prove2Me	An open platform: a paper becomes a *mission*, and anyone's agent can contribute proofs.	Yes: prove2.me.
Tau Ceti	A Lean library downstream of Mathlib: humans write the roadmaps, AI writes the code.	Through its roadmaps.
Gauss	Math Inc's autoformalization agent.	Early access only.
Aristotle	Harmonic's agent. Its current version verifies code (Verilog, C, Rust); it "doesn't verify pure mathematics or Lean".	Not for Lean, today.

Product pages and documentation, read on 2 November 2026. This changes fast: check before you rely on it.

A coding agent

- It reads your files, writes Lean, runs Lean, and reads the errors. Then it tries again.
- Lean's error messages are its feedback. That is why it works for proofs: every attempt gets an exact answer.
- It can work for a long time without you. You check the result.



Setting it up for Lean

1. A Lean project with Mathlib (session 1).
2. Claude Code, in the terminal or in VS Code.
3. **lean-lsp-mcp**: gives the agent what you see in the infoview. In the project folder:

Terminal

```
claude mcp add lean-lsp uvx lean-lsp-mcp
```

github.com/oOo0oOo/lean-lsp-mcp. Other coding agents work in the same way.

What the agent can then do

- See the goal at any line, and the errors.
- Search Mathlib: LeanSearch, Loogle, local search.
- Check which axioms a proof uses.

Pin the statement

live in Lean

```
-- You write this, and the agent may not
    edit it.
def Statement : Prop :=
  ∀ n : ℕ, ∑ i ∈ Finset.range (n + 1), i
    = n * (n + 1) / 2

-- The agent writes this.
theorem main : Statement := by
  intro n
  rw [Finset.sum_range_id,
    Nat.add_sub_cancel,
    mul_comm]
```

- Put the statement in a `def` that the agent may not edit, and ask for `theorem main : Statement`.
- If the agent changes the statement to make it easier, you see it in the file history.
- This is what the comparator did for FLT, at scale.

Instructions for the agent

A rules file for the project

- Never edit the file with `Statement`.
- Never add `axiom`, `sorry` or `native_decide`.
- Search Mathlib before proving a lemma.
- Before writing Lean, write the plan: the lemmas, in English.
- If a step fails, stop and report it. Do not change the statement.

- Claude Code reads such rules from a file `CLAUDE.md` in the project.
- Ask for the plan first. A wrong plan costs you one paragraph; a wrong proof costs an hour.
- At the end, ask what it used from Mathlib, and what it could not prove.

Prove2Me

- A researcher launches a **mission**: a headline goal, the definitions it needs, and milestone lemmas.
- Humans audit this core before the mission is released, “and audit nothing beyond it”. A read-back step translates the Lean into LaTeX for comparison with the source.
- Agents split proofs into *proof sketches*: each child lemma “immediately becomes a new problem on the platform”.
- Proved results go into a shared library that later proofs can import.

> The human work is the statements. Everything after that is open to anyone’s agent.

S. Chen, K. Marwaha, X. Lu, H. Yuen, T. Peng, *Prove2Me*, arXiv:2608.28433 (August 2026).

Tau Ceti

- "An AIs-welcome Lean library downstream of Mathlib: AI handle the implementation and review, humans write the roadmaps and review rubrics."
- A **roadmap** is a `README.md`, "the definitive specification of its area", usually with suggested Lean target statements in `Suggested.lean`.
- Roadmap changes go through human review. Anyone can propose one.
- Incubated by the Lean FRO and the Mathlib Initiative.

> A roadmap is a set of statements, written and checked by people. The proofs are left to AI.

github.com/TauCetiProject/TauCeti and TauCetiRoadmap.

Checking what comes back

Lean accepted it. Now what?

Lean accepting a file means less than it seems. The file may still contain:

- a **changed statement**: easier than the one you asked for;
- a `sorry`, hidden inside a longer proof;
- a new **axiom**: the result, simply assumed;
- `native_decide`: a computation trusted to the compiler, not checked by the kernel;
- hypotheses that cannot hold, or that assume the conclusion (session 2).

> None of these is an error. You find them by looking.

Spot the problem (1)

```
axiom helpful_fact : (2 : ℕ) + 2 = 5

theorem looks_fine : (2 : ℕ) + 2 = 5 :=
  helpful_fact

#print axioms looks_fine
-- depends on axioms: [helpful_fact]
```

Spot the problem (1)

```
axiom helpful_fact : (2 : ℕ) + 2 = 5

theorem looks_fine : (2 : ℕ) + 2 = 5 :=
  helpful_fact

#print axioms looks_fine
-- depends on axioms: [helpful_fact]
```

- `axiom` asserts a statement without proof. Here a false one, and Lean accepts `looks_fine`.
- `#print axioms` lists it. Anything beyond the three standard axioms needs an explanation.

Spot the problem (2)

```
theorem almost (x : ℝ) (hx : 0 < x) :
    Real.log x ≤ x - 1 := by
  have key : ∀ y > 0, Real.log y ≤ y - 1 := by
    sorry
  exact key x hx

#print axioms almost
-- depends on axioms:
--   [propext, sorryAx, Classical.choice, Quot.sound]
```

Spot the problem (2)

```
theorem almost (x : ℝ) (hx : 0 < x) :
    Real.log x ≤ x - 1 := by
  have key : ∀ y > 0, Real.log y ≤ y - 1 := by
    sorry
  exact key x hx

#print axioms almost
-- depends on axioms:
--   [propext, sorryAx, Classical.choice, Quot.sound]
```

- The key step is `sorry`. Lean warns, but in a long file the warning is easy to miss.
- `#print axioms` shows `sorryAx`: the proof is not finished.

Spot the problem (3)

```
theorem big_prime : Nat.Prime 1000003 := by
  native_decide

#print axioms big_prime
-- depends on axioms: [propext, Classical.choice,
--   Quot.sound, big_prime._native.native_decide.ax_1_1]
```

Spot the problem (3)

```
theorem big_prime : Nat.Prime 1000003 := by
  native_decide

#print axioms big_prime
-- depends on axioms: [propext, Classical.choice,
--   Quot.sound, big_prime._native.native_decide.ax_1_1]
```

- `native_decide` runs compiled code and trusts its answer. The kernel does not check it.
- It shows up as an axiom you have never heard of. Here `norm_num` proves the same statement, and the kernel checks it.

What a clean proof looks like

```
theorem log_le (x : ℝ) (hx : 0 < x) :
    Real.log x ≤ x - 1 :=
  Real.log_le_sub_one_of_pos hx

#print axioms log_le
-- depends on axioms:
--   [propext, Classical.choice, Quot.sound]
```

- The same statement as in (2), proved by a Mathlib lemma.
- Only `propext`, `Classical.choice` and `Quot.sound`: the three standard axioms of Lean and Mathlib.

Checking an AI's proof

live in Lean

1. **The statement is unchanged.** It is pinned, and the file history shows no edit to it.
2. **It builds**, with no errors and no `sorry` warnings.
3. **Axioms.** `#print axioms main` shows only the three standard ones.
4. **Warnings.** An unused hypothesis is a warning sign (session 2).
5. **The statement still passes the tests**: the hypotheses can hold, and it reads back as the paper.

> Steps 2 and 3 are mechanical. Steps 1 and 5 are yours.

At home

At home

`Exercises3.lean`, on your laptop.

1. Let an agent prove it: five statements, one at a time, with the rules from the slides. Check every proof it hands back.
2. Audit: seven proofs “from an agent”, all accepted by Lean. Which ones prove the paper statement?
3. Your statement from session 2: pin it, and let an agent try.

- No agent? Prove part 1 yourself, with `exact?` and the tactics from session 1.
- A coding agent needs an account, usually paid. The setup sheet has the steps.
- If the agent gets stuck on your statement, write down where.

Next time: your own theorem

- Two or three of your statements, formalized live: statement, tests, proof, audit.
- A template for doing your own at home.
- Where to go after the course: the Lean Zulip, contributing to Mathlib, larger projects.

> Before next time: send me your statement, if you have not yet. Short and elementary works best.

Sources

- Anthropic, *Formalizing Fermat's Last Theorem*, 4 September 2026.
- K. Buzzard, Xena blog: 20 July 2026 (unit distances); guest post, 16 October 2026.
- Logical Intelligence, *Aleph Prover* blog post (unit distances).
- Math, Inc.: *Introducing Gauss*; *Sphere packing*.
- S. Chen et al., *Prove2Me*, arXiv:2608.28433; prove2.me.
- Tau Ceti: github.com/TauCetiProject.
- lean-lsp-mcp: github.com/oOo0oOo/lean-lsp-mcp. Aristotle: aristotle.harmonic.fun.